



CIRCUITOS DIGITALES Y MICROCONTROLADORES 2025

Facultad de Ingeniería
UNLP

Introducción a los Sistemas Embebidos

Ing. José Juárez

Introducción a los Sistemas Embebidos

- Definición de **Sistema Embebido*** (*Embedded System*):

Es el nombre genérico que reciben los equipos electrónicos que incluyen **un procesador de datos**, pero que a diferencia de una PC, están diseñados para satisfacer **una función específica**, como en el caso de un reloj, un reproductor de MP3, un router wifi, un joystick de Play, o pueden ser componentes de un sistema de control de un electrodoméstico, de un automóvil, de un avión, etc.

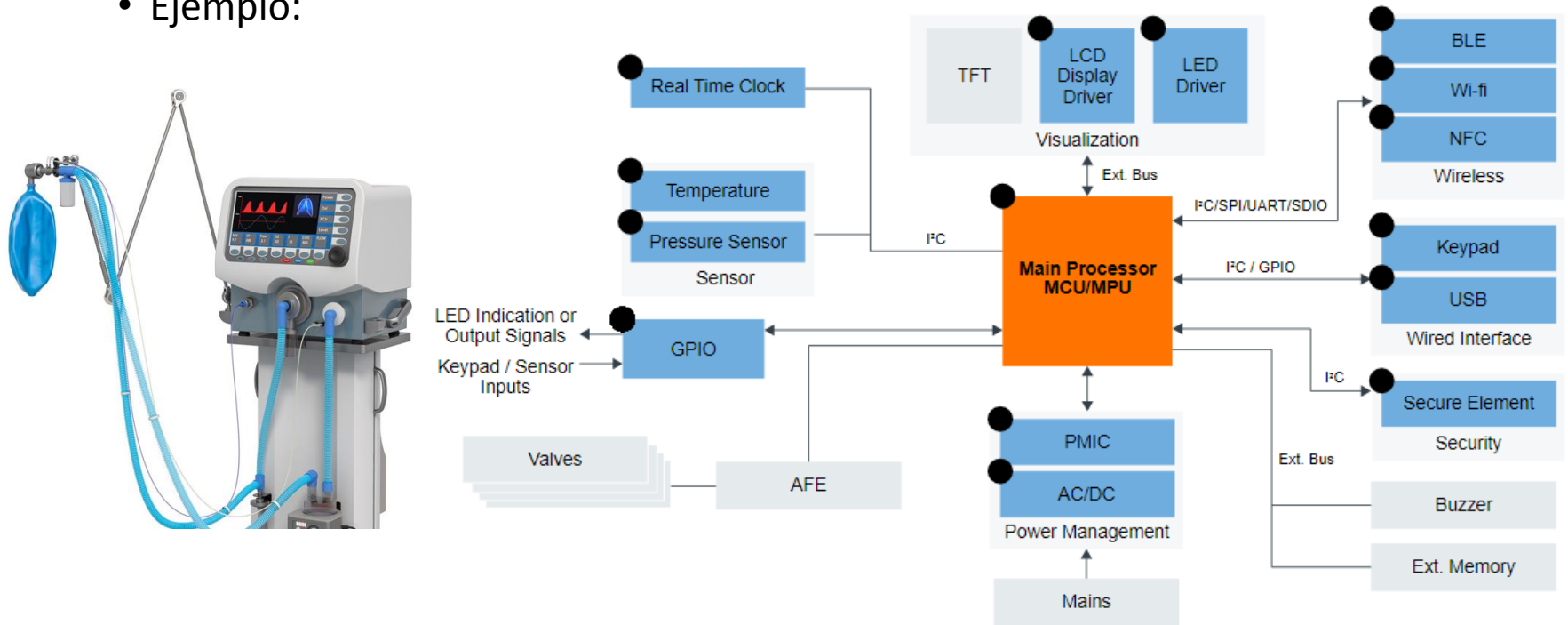


Es un sistema electrónico que está contenido o **“embebido”** dentro de un equipo completo que incluye además, por ejemplo, partes mecánicas o electromecánicas.

*SASE: SIMPOSIO ARGENTINO DE SISTEMAS EMBEBIDOS (<https://sase.ar/>)

Introducción a los Sistemas Embebidos

- El **cerebro** de un sistema embebido es típicamente un **microprocesador** o **microcontrolador**, y su diseño está optimizado para reducir su tamaño, el consumo, costo, y maximizar su confiabilidad y desempeño.
- Ejemplo:



[Respirador artificial \(NXP Design\)](#)

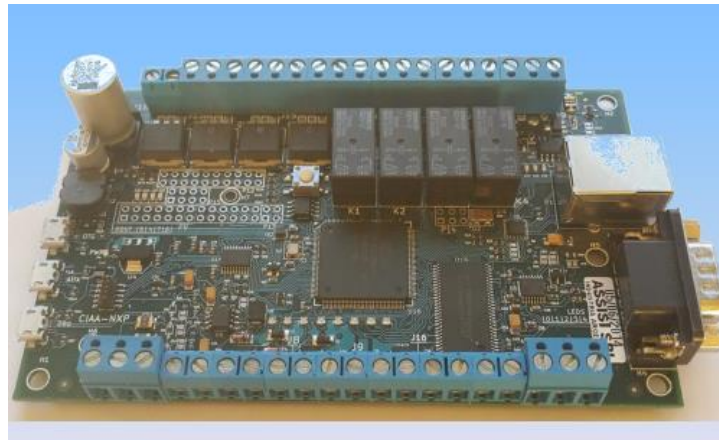
- El diseño de sistemas embebidos es un motor clave de la industria y del desarrollo tecnológico, y es un campo que en los últimos años ha crecido notablemente en la Argentina.

Componentes de un Sistema Embebido

- Componentes de Hardware:

- Microprocesador, Microcontrolador, DSP, FPGA, ASIC, SoC, otros...
- Memorias de almacenamiento volátil y no volátil
- Periféricos analógicos y digitales
- Componentes Electrónicos Activos (diodos, transistores) y pasivos (resistencias, capacitores, inductores)
- Interfaces Eléctricas (conectores)
- Placa de circuito Impreso (PCB)

Uno de sus objetivos fue proveer a la industria local de un dispositivo Open Source que se comporte como un PLC (Programmable Logic Controller)



CIAA

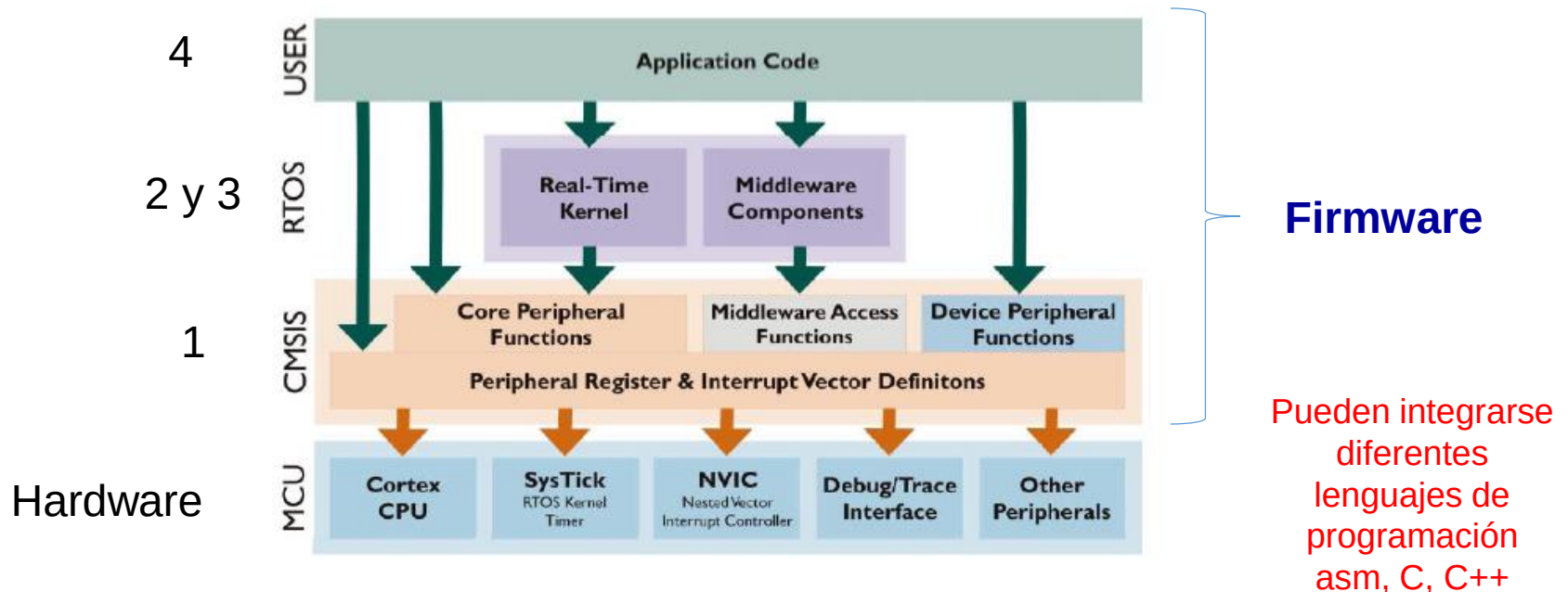
Computadora
Industrial
Abierta
Argentina

<http://www.proyecto-ciaa.com.ar/>

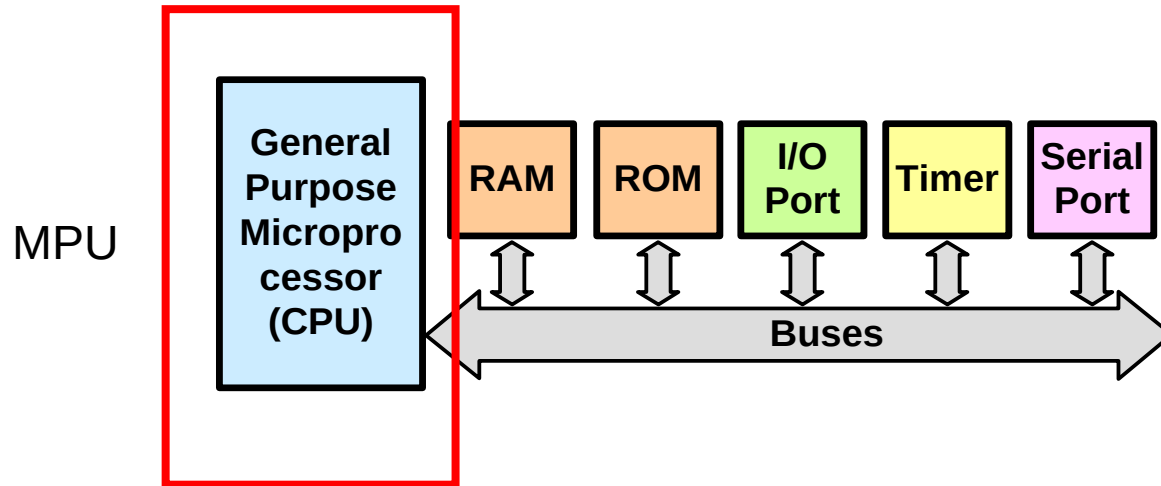
Componentes de un Sistema Embebido

- Componentes de Software:

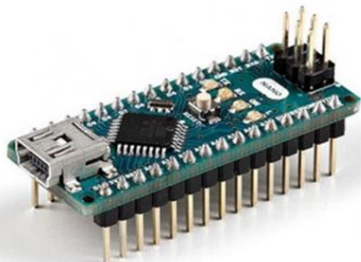
- 1-Capa de abstracción de hardware y manejadores de dispositivos (Drivers)
- 2-Capa de componentes intermedios como pila de protocolos
- 3-Capa de Planificación y ejecución de tareas en tiempo real (RTOS)
- 4-Capa de la aplicación específica



Microprocesador (MPU) o Microcontrolador (MCU)

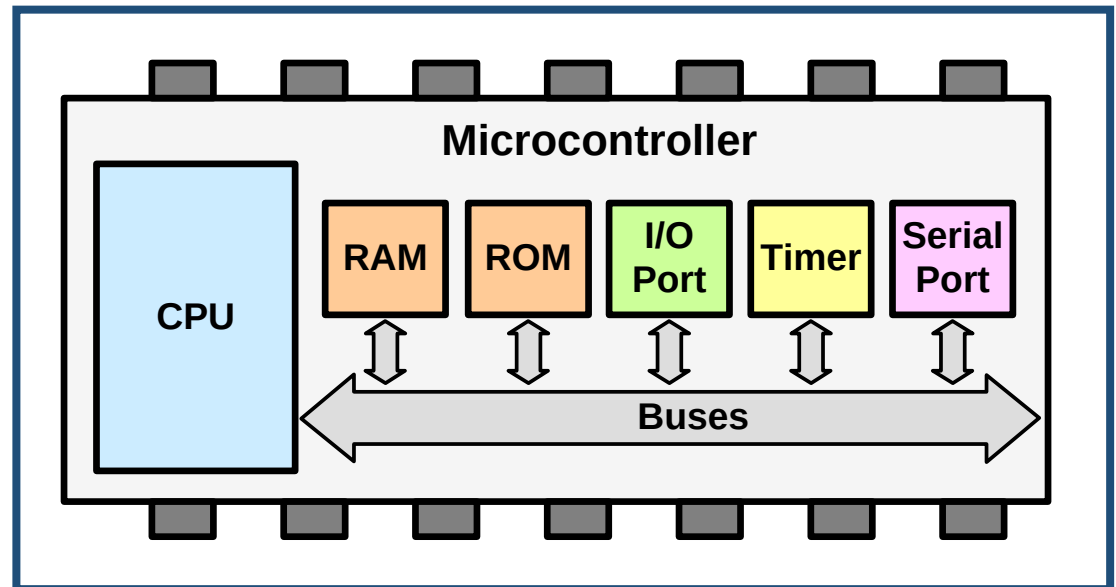


Quad core Cortex-A72
(ARM v8) 64-bit SoC @ 1.5GHz
(Raspberry Pi 4)



Atmega328p
8-bits AVR @ 16MHz
(Arduino Nano)

MCU



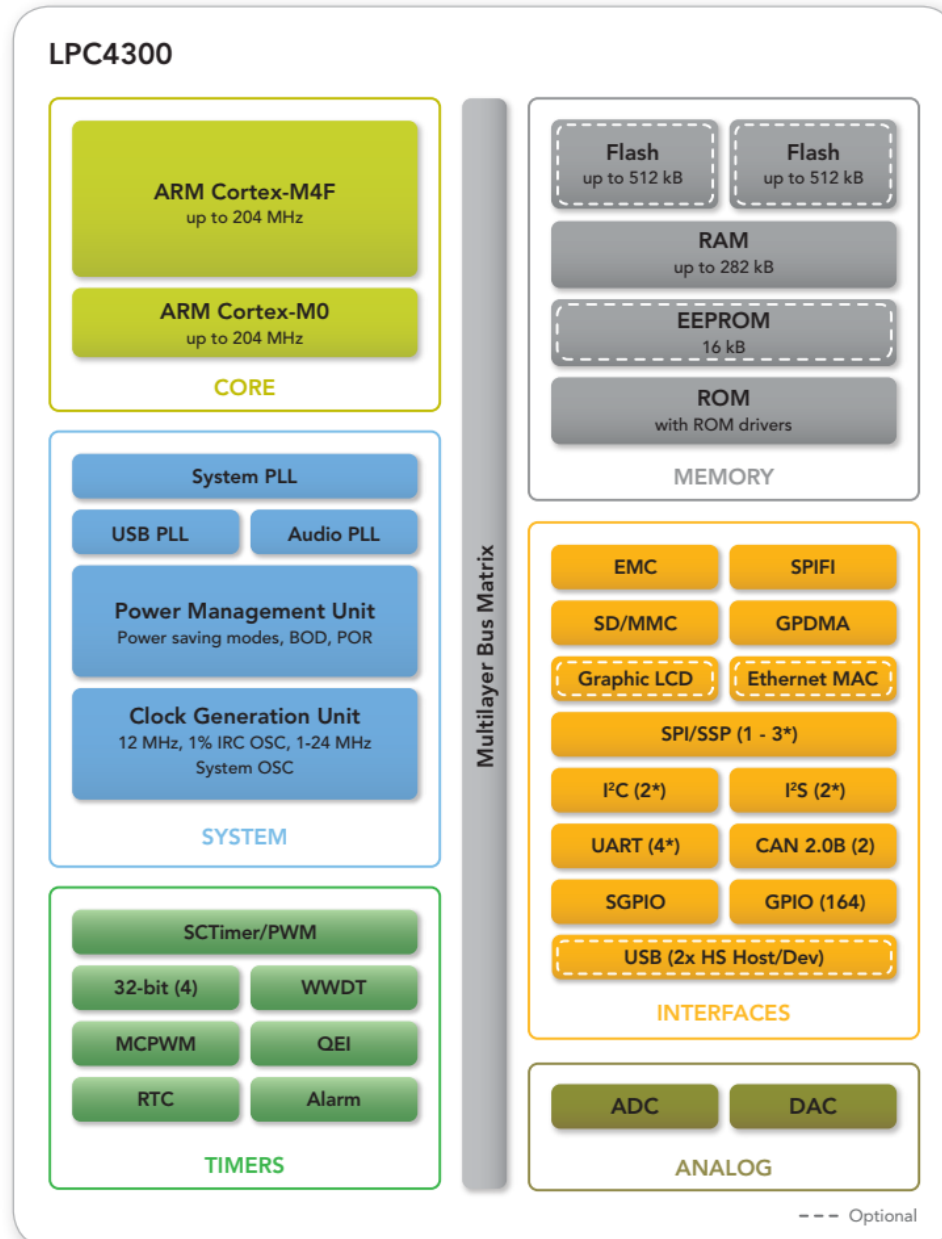
Microprocesador (MPU) o Microcontrolador (MCU)

- μ P(MPU): Unidad de procesamiento de propósito general
 - gran capacidad de computo, 16 , 32 y 64 bits, FPU (Floating Point Unit), reloj de GHz,
 - múltiples núcleos, con diferentes niveles de memoria cache L1,L2
 - Poseen MMU (Memory Management Unit) para soporte a sistemas operativos
 - sin periféricos específicos integrados o solo los necesarios,
 - Ej: Intel 8086, Intel Pentium, Sun SPARC, PowerPC, MIPS, ARM7, ARM9,ARM11, ARM-Cortex A
- μ C (MCU): Sistema de procesamiento completo destinados a aplicaciones específicas
 - moderada capacidad, 8, 16, 32bits, ALU de punto fijo, reloj <500MHz
 - con periféricos I/O integrados en el mismo chip.
 - Encapsulados entre 6 y 100 terminales,
 - Bajo consumo, bajo costo.
 - Ej: Intel 8051, Freescale HC08, Microchip PIC, Atmel AVR, Texas MPS430, Freescale HC12, Microchip PIC32, Atmel AVR32, ARM-Cortex M.
- En la actualidad esta diferencia se hace cada vez menos notoria en la gama de arquitecturas de 32bits
 - MCU 32bits, ALU con FPU, núcleos asimétricos, Arquitectura Superescalar

Otras tecnologías utilizadas en Sistemas Embebidos

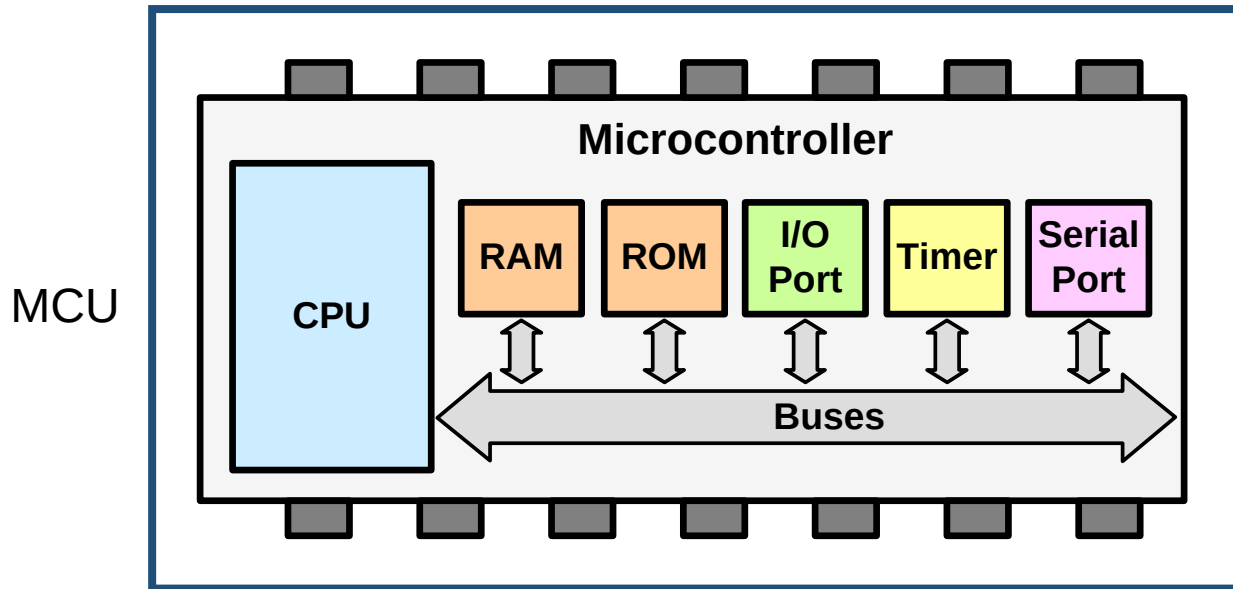
- **ASIC:** Application Specific Integrated Circuit
Diseñado específicamente para una aplicación particular (Digital, Analógico, mixto).
Provee la mejor performance pero el costo de desarrollo es muy elevado.
Restringido a producciones de mucho volumen
- **DSP/DSC:** Digital Signal Processor / Digital Signal Controller
16 o 32 bits, Unidad de Punto flotante o Punto fijo,
Instrucciones optimizadas para operaciones matemáticas específicas.
Periféricos específicos de acuerdo a la aplicación.
Utilizado para procesamiento de señales de Audio, Video, comunicaciones...
- **FPGA:** Field Programmable Gate Array
Circuitos lógicos reconfigurables
Diseños específicos muy optimizados para computo de alta performance
Diseños propietarios (IP)
- **SoC:** System on Chip
Procesador (de 8, 32 o 64bits) + FPGA para diseño de periféricos
Soluciones específicas o de altas prestaciones.

Arquitectura de un microcontrolador actual



Arquitectura de un microcontrolador

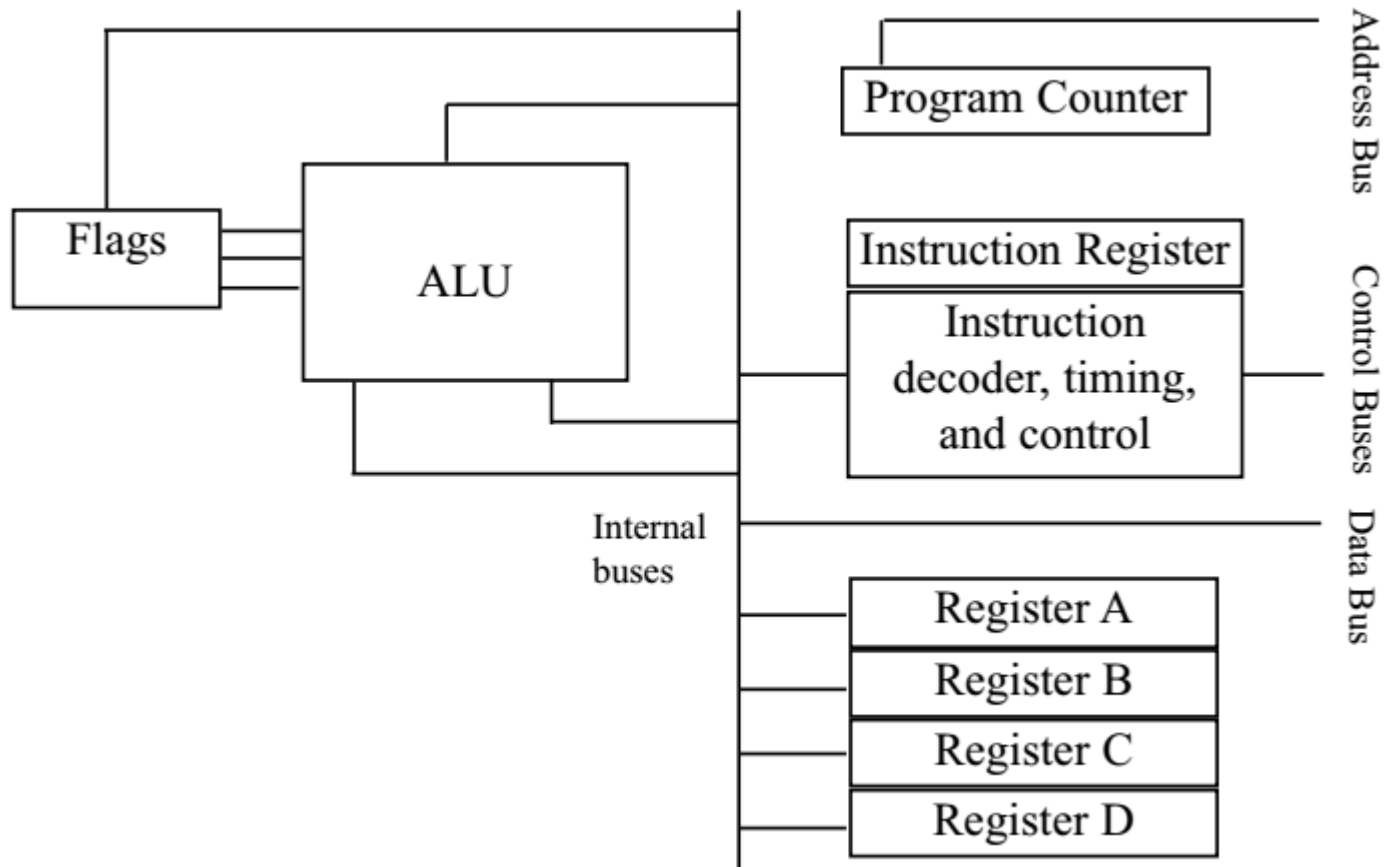
Mejor volvamos a la versión simplificada:



Repasemos los componentes y características más relevantes

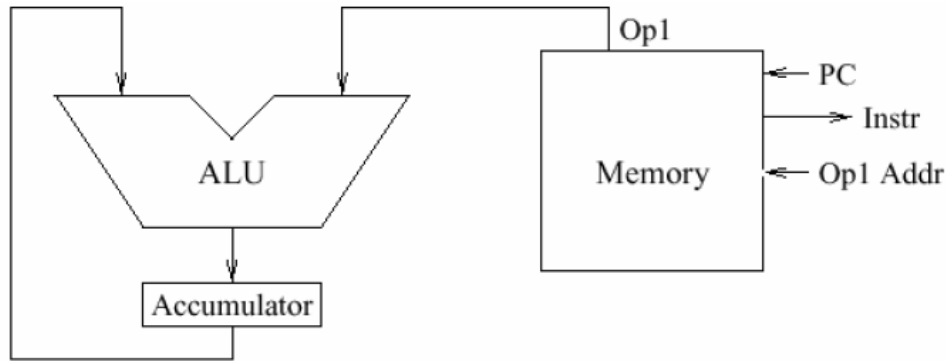
CPU (Central Processing Unit)

- Diagrama básico de una CPU



Arquitecturas de CPU

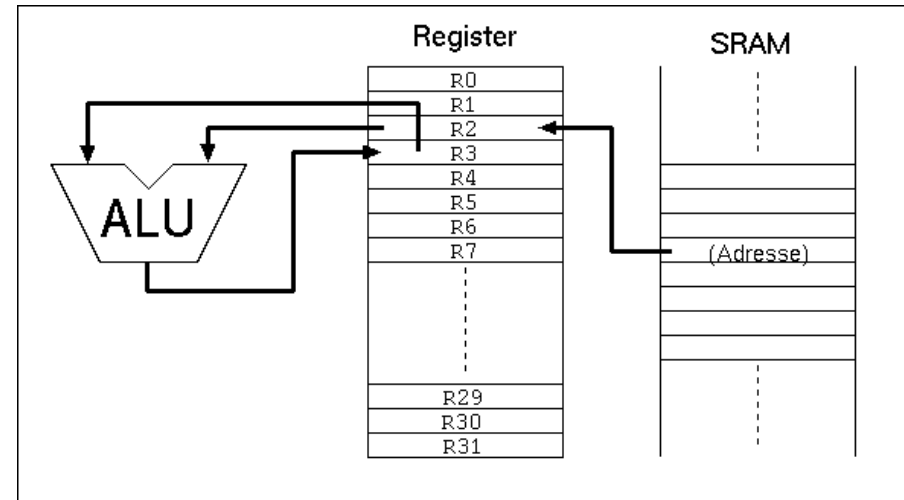
- CPU Basadas en Acumulador



$Acc + Op1 \rightarrow Acc$

- Ejemplos: NXP S08
- Otro ejemplo: Microchip PIC
- Ambos de 8 bits

- CPU Basada en Registros



$R2 + R3 \rightarrow R3$

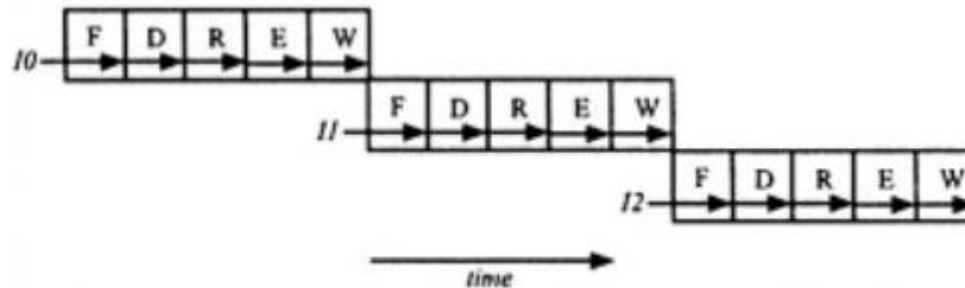
- Ejemplos:
 - Atmel AVR : 32 registros de 8 bits
 - ARM cortex-M: hasta 16 registros de 32 bits
 - RISC-V: 32 registros de 32 bits

Ejecución secuencial de instrucciones

- El contador de programa (PC) indica la dirección de memoria de la siguiente instrucción a ejecutar.
- Supongamos que la ejecución de una instrucción requiere de 5 etapas:

Fetch (F) : Decode (D) : Read Register File (R) : Execute (E) : Writeback (W)

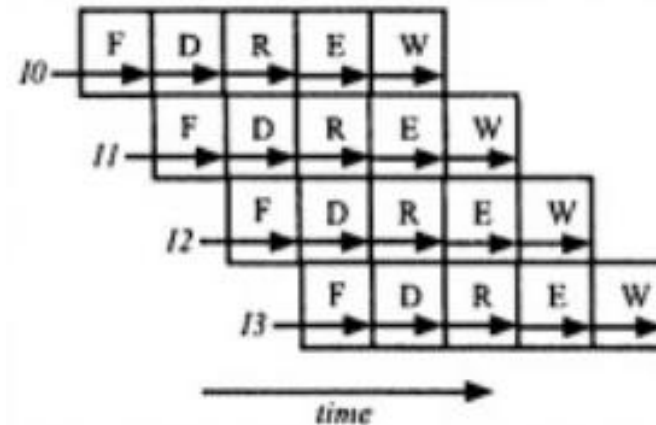
- La siguiente figura muestra el flujo de ejecución de instrucciones en un procesador genérico secuencial



- El procesador requiere completar la ejecución de una instrucción antes de proceder con la siguiente
- Ejemplo MCU: Intel 8051, NXP S08

Ejecución de instrucciones con pipeline

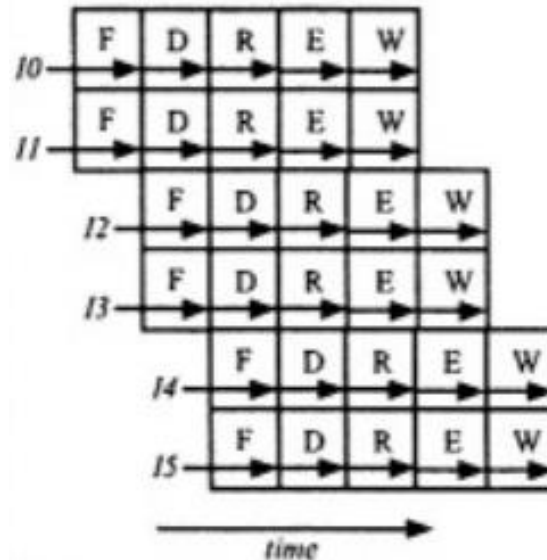
- La siguiente figura muestra el flujo de ejecución de instrucciones en un procesador genérico con pipeline



- El procesador no espera completar la ejecución de una instrucción, si no que comienza con la siguiente a medida que las etapas van completando su función ciclo a ciclo
 - *qué pasa en el caso de ramificaciones?*
 - *qué pasa si una instrucción usa un dato que modificó la previa?*
- Ejemplos MCU:
 - Atmel AVR (2-etapas),
 - ARM cortex M (3-etapas c/predicción de salto)

Arquitectura Superescalar

- La siguiente figura muestra el flujo de ejecución de instrucciones en un procesador superescalar de 2 vías.



- El procesador ejecuta más de una instrucción por etapa del pipeline.
- Ejemplo MCU: ARM cortex M7

Clasificación según el conjunto de instrucciones

RISC: Reduced Instruction Set Computer

- Pocas instrucciones, Operaciones Simples
- Ejecución en su mayoría en 1 ciclo de reloj
- Longitud de instrucción fija
- Pocos modos de direccionamiento a memoria
- Ejecución de instrucciones en Pipeline
- Opera sobre un gran número de registros no especializados (datos o direcciones)
- Las instrucciones de Load/Store son las únicas que acceden a memoria
- Requiere mayor cantidad de instrucciones para un código determinado comparado con la arquitectura CISC
- Por esta misma razón el código binario ocupa mayor cantidad de memoria
- El compilador es el encargado de realizar las tareas complejas
- Ejemplo: RISC-V, ARM cortex-M, AVR, PIC

Clasificación según el conjunto de instrucciones

CISC: Complex Instruction Set Computer

- Muchas Instrucciones , operaciones Complejas (DSP)
- Ejecución en varios ciclos de reloj
- Longitud de instrucción variable
- Gran número de modos de direccionamiento a memoria
- Opera sobre un número acotado de registros
- Además de las instrucciones de Load/Store, posee otras instrucciones que acceden a memoria
- Requiere menor cantidad de instrucciones para un código complejo comparado con la arquitectura RISC (menor espacio de memoria)
- Requiere compiladores mas complejos
- Ejemplo: Intel 8051, x86, Motorola HC08, PowerPC

Estructura de Buses

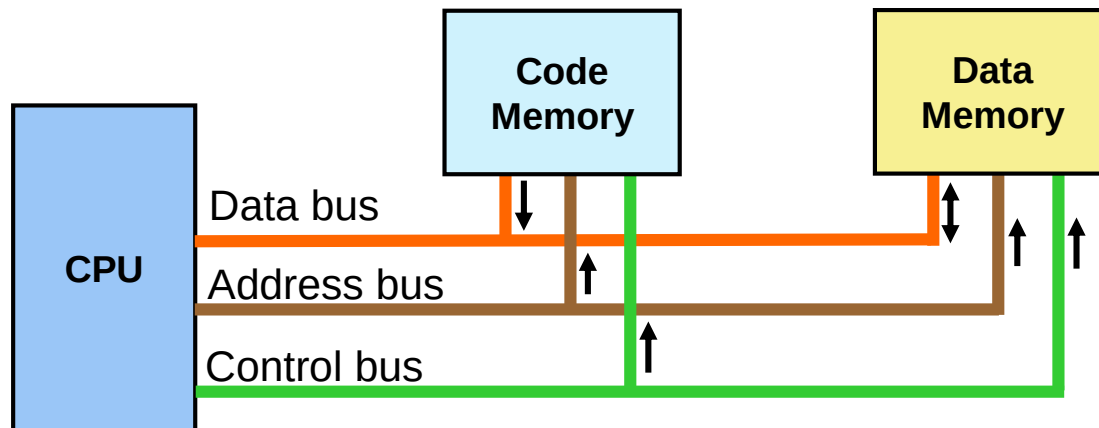
- **Bus de Direcciones:** Controlado por la CPU para direccionar los demás subsistemas. Es unidireccional. El tamaño determina la cantidad de memoria direccionable: $N \text{ bits} \Rightarrow 2^N$, direcciones.
- La mayoría de los MCU de 8 o 16 bits poseen bus de direcciones de 16 bits lo que permite direccionar $65536=64\text{KB}$ o 64KiB posiciones de memoria distintas. Los MCU de 32 bits poseen bus de direcciones y datos de 32 bits con lo que es posible direccionar hasta 4GB.
- **Bus de Datos:** comunicación paralela entre los subsistemas y la CPU. Es bidireccional. Está definido por la capacidad de operación de la ALU y la memoria de datos (data path).
- **Bus de Control:** líneas de control para el acceso a los subsistemas. Puede ser unidireccional o bidireccional. Ejemplos: señales de CS: chip select, señales de R/W: read/write, señal de Reloj.

Organización de la memoria: En su gran mayoría, los MCU no poseen Unidad de manejo de Memoria (MMU - Memory Management Unit) y tratan a todo el espacio direccionable como una única entidad (mapa de memoria unificado). La ausencia de páginas o bancos de memoria hace a las arquitecturas más simples de usar.

Estructura de Buses

Von Neumann

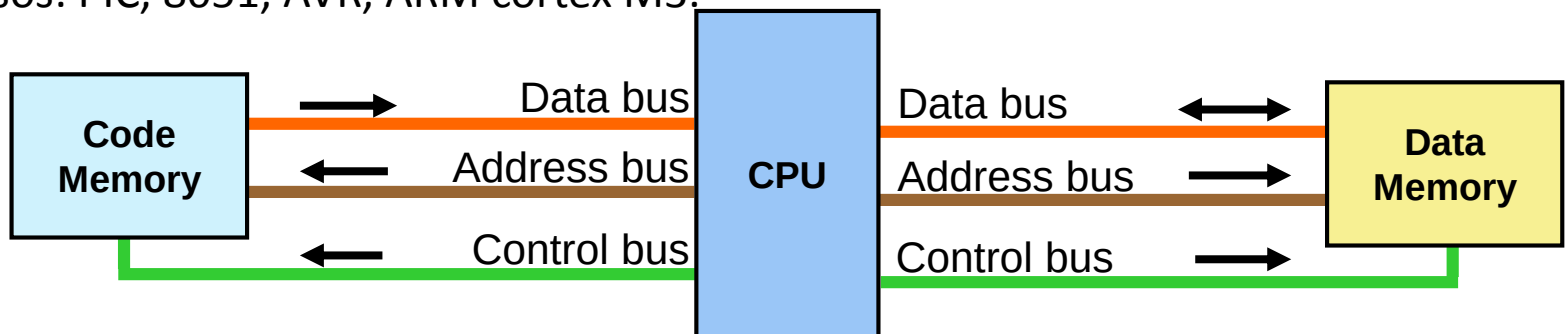
- El sistema contiene un solo bus de direcciones que se utiliza para acceder a todo el espacio de memoria disponible (datos y programa).
- Los datos y las instrucciones comparten el mismo bus.
- Es menos eficiente ya que puede requerir de varios ciclos para acceder a instrucciones complejas (varios bytes) sin embargo el sistema es más simple, una misma instrucción gestiona memoria de datos y de programa y no hay diferencia entre acceder a una constante o un dato.
- Casos Motorola HS08, Texas MSP430



Estructura de Buses

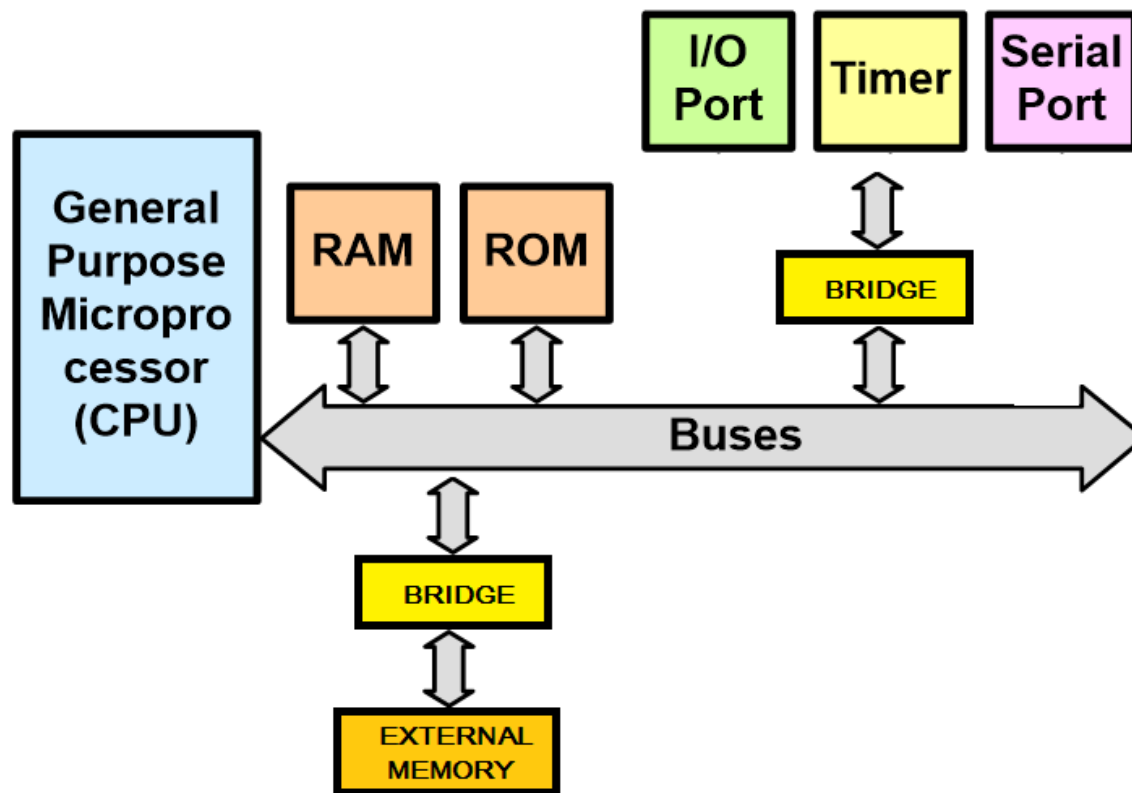
Hardvard

- La memoria de programa y la memoria de datos se tratan como subsistemas diferentes. Cada uno posee su propio bus de direcciones y su propio bus de datos.
- Las principales ventajas son:
 - Permite acceso simultaneo a instrucciones y datos (la CPU puede leer un operando al mismo tiempo que esta leyendo la siguiente instrucción)
 - Cada subsistema se optimiza por separado. Por ejemplo PICxx posee un bus de datos de 8 bits, con 9 bits de direcciones y una memoria de programa de 14 bits (una instrucción completa por palabra) con 13 bits de direcciones.
- Una desventaja se presenta con el acceso a datos constantes (tablas de lookup, cadenas de caracteres). Se requiere de instrucciones especiales para manejo de datos en la memoria de programa o el mapeo de un bloque de memoria de programa en el espacio de datos.
- Casos: PIC, 8051, AVR, ARM cortex M3.



Estructura de Buses

Bridges (puentes)

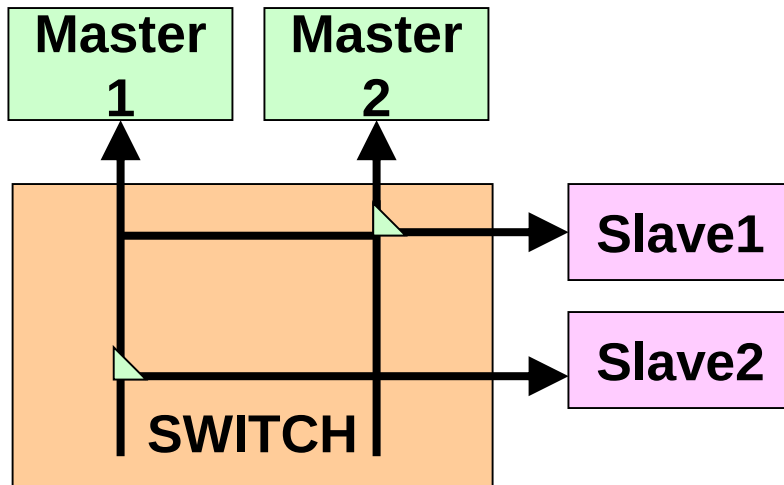


- Un bridge es un intermediario entre dos buses
- Permite acceso simultáneo a diferentes memorias y E/S
- Estos pueden ser similares o responder a distintos mecanismos de control
- Y a ambos lados del bridge suceden transacciones al mismo tiempo
- Pueden tener distintos tamaños de datos y modos de almacenamiento temporario
- Son comunes los bridges a periféricos y a memorias masivas

Puede ser Harvard, Von Neumann, u otras

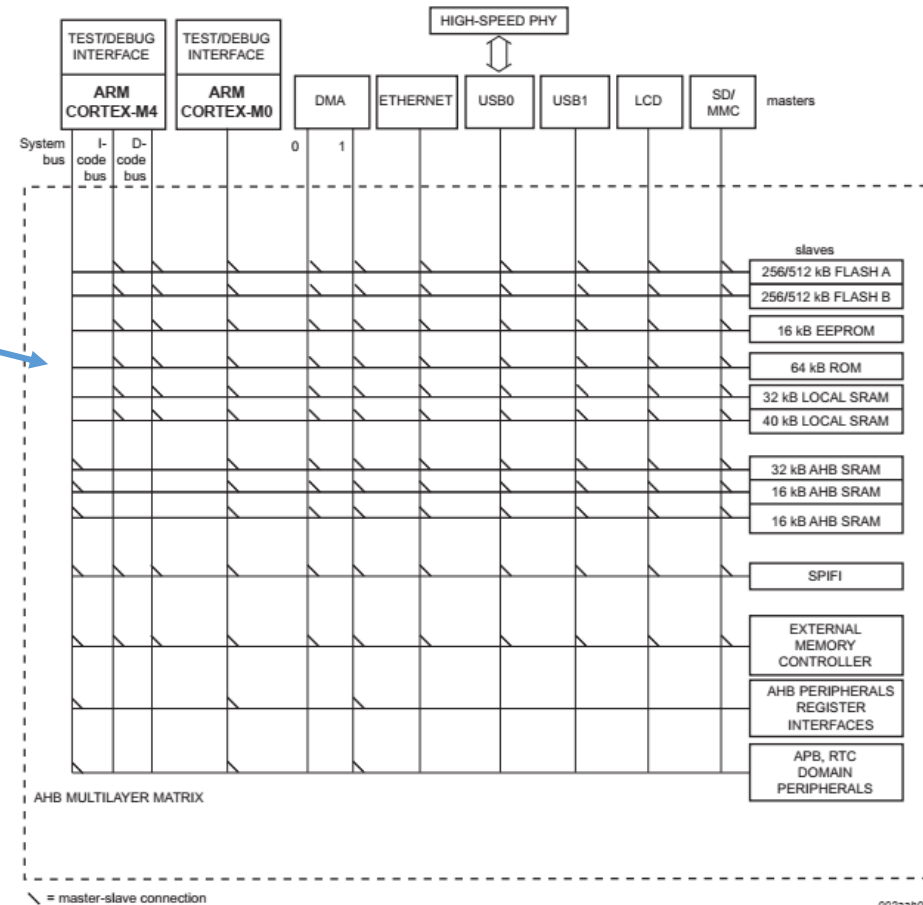
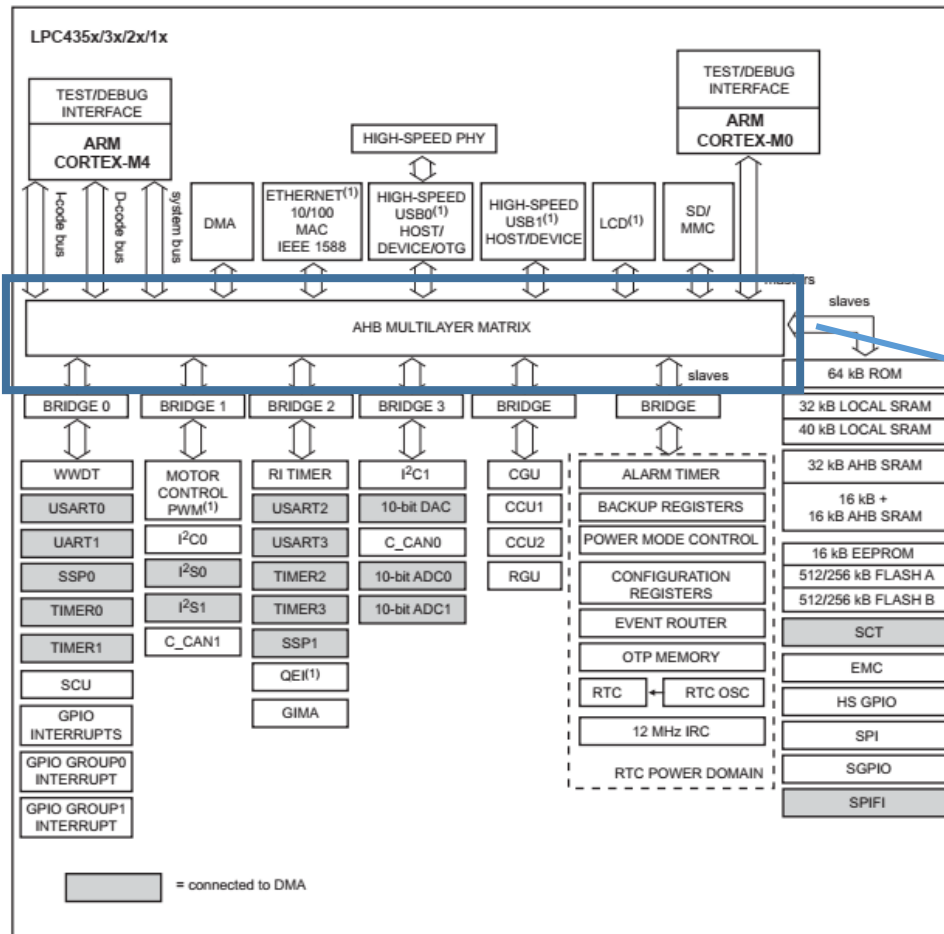
Estructura de Buses

Switches



- Puede convenir poner varios procesadores en un mismo sistema
- Si varios procesadores desean realizar transacciones deben esperar su turno para evitar colisiones (*contention*)
- Un *switch* permite que varias transacciones con orígenes y destinos diferentes puedan ser realizadas simultáneamente
- Aumenta la tasa de intercambio de datos, a costa de mayor complejidad del circuito
- Es necesario arbitrar los accesos a recursos comunes

Ejemplo: arquitectura ARM Cortex M



002aah090

Exploraremos esta arquitectura en Taller de Proyecto I

Tipos de Memorias de datos

RAM (Random Access Memory)

- Memoria volátil, provee almacenamiento temporal de datos (variables, pilas) mientras tenga alimentación.
- Cualquier posición de la memoria se puede acceder para leer o escribir con el mismo tiempo de acceso.
- La escritura sobrescribe el dato anterior
- Encontramos mayoritariamente 2 tipos:
 - SRAM (static RAM): que significa que mantiene su contenido incluso si se detiene la señal de reloj. Estas memorias poseen celdas basadas en F.F. de 6 transistores (1bit), lo que hace difícil integrarla en gran volumen.
 - DRAM (Dynamic RAM): La memoria RAM de una PC es del tipo DRAM. Cada elemento contiene un transistor y un capacitor para almacenar 1 bit, pero requiere de un refresco periódico para mantener su valor. Esto es impráctico en los MCU.

EEPROM (Electrically Erasable Programmable ROM):

- Programable y borrable eléctricamente. Mediante un circuito controlador se puede borrar de a un byte, de a bloques o completa. Esta característica limita la densidad de integración. Permite almacenar datos no volátiles y constantes.

Tipos de Memoria de programa

ROM :

- Es realmente una memoria de solo lectura y no volátil. Los datos son escritos en el proceso de fabricación del dispositivo mediante máscaras y fotolitografía.
- Muchos fabricantes de MCU ofrecen dispositivos con este tipo de memoria a pedido, anexando bootloader, bibliotecas de código de protocolos específicos, etc.

FLASH:

- Se pueden escribir y borrar eléctricamente más rápido que sus antecesoras EEPROM. Es la tecnología dominante en la actualidad.
- En el caso de los MCU la memoria FLASH contiene un sistema de programación que permite su programación sin retirarla del circuito (In Circuit Programming).
- La gran diferencia con la EEPROM es que posee mayor capacidad de integración aunque no se puede borrar de bytes individuales, solo por bloques o total.
- Las memorias FLASH y EEPROM poseen un número de ciclos finito de escritura y borrado. Actualmente del orden de los 10.000 a 100.000 ciclos.
- La memoria FLASH en un MCU son de tecnología NOR FLASH que permite acceso aleatorio y es diferente a la utilizada en dispositivos de almacenamiento masivo como Discos, pendrives, tarjetas SD que son tecnología NAND y se acceden de manera secuencial a alta velocidad.

Periféricos integrados

- **Entrada Salida (I/O Ports)**

- Los puertos de entrada / salida nos permiten ejercer acciones digitales sobre el medio exterior o leer estados o condiciones de dispositivos externos.

- **Periféricos de comunicaciones**

- UART
- SPI
- I2C
- USB
- ETHERNET

- **Periféricos analógicos**

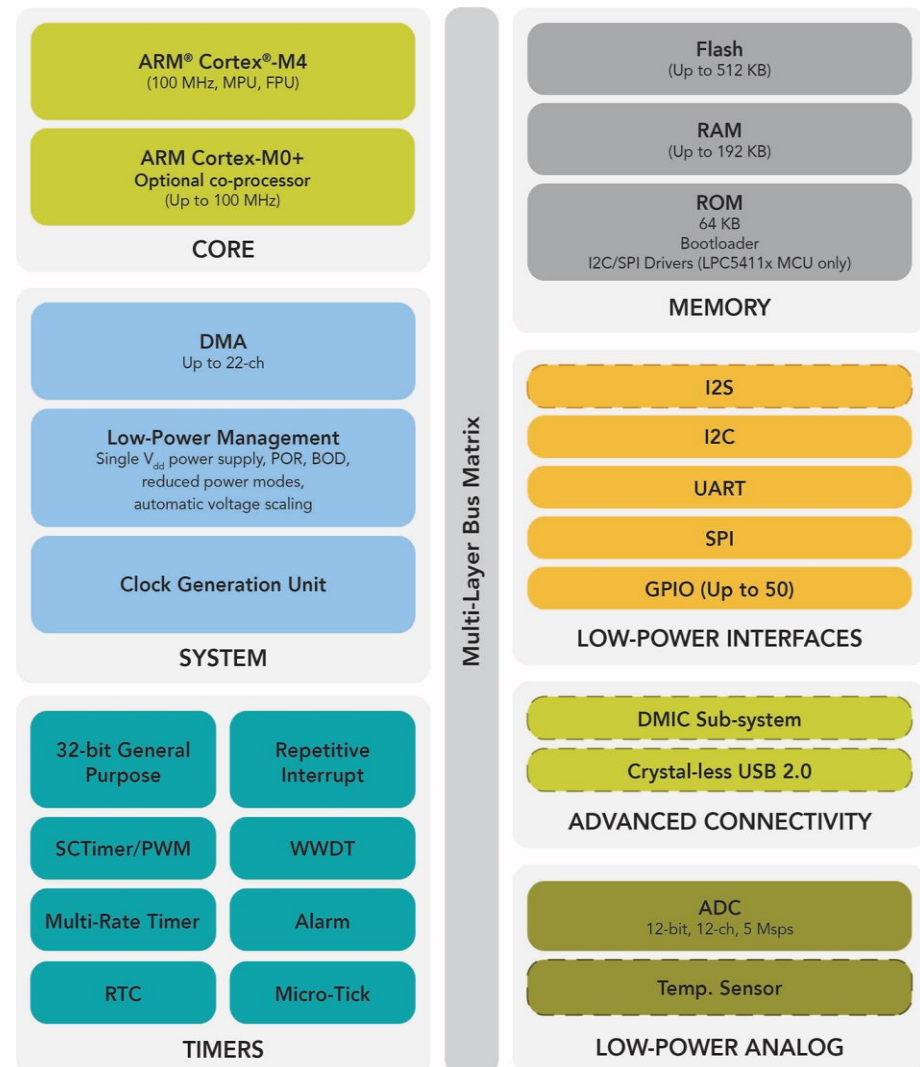
- ADC
- DAC
- COMPARADOR

- **Periféricos de temporización y control**

- TIMERS
- PWM
- RTC

- **Periféricos específicos**

- CRYPTO (AES/DES)
- LCD /KEYPAD Controller
- External Memory Controller



Comparativa*

DMIPS = Dhrystone MIPS

- Performance:

	Procesador	DMIPS (aprox.)	(DMIPS/MHz)	coremark/MHz
MCU	ATmega32A (AVR, 8 bits, 16 MHz)	5,9	0,37	0.53
	ARM Cortex-M0 a 50 MHz	45	0,9	
	ARM Cortex-M3 a 100 MHz	125	1,25	1,5
MPU	ARM Cortex-A8 a 1 GHz	2000	2	
	Intel Atom-Z530 a 1,6 GHz	3200	2	
	Intel Core i7 EE-990x (6 cores) a 3,46 GHz	159000	45	

- Consumo:

<https://www.eembc.org/coremark/scores.php>

	CPU	Flash	Pin Count	Active ⁽¹⁾	Static ⁽²⁾ Typ @ 25°C
Atmel SAM D20	32-bit ARM Cortex-M0+	16 – 256kB	32 – 64	140 µA/MHz	2 µA
Freescall Kinetis K20	32-bit ARM Cortex-M4	32 – 160kB	32 – 64	280 µA/MHz	1.9 µA
Atmel ATmegaX8PA	8-bit AVR	4 – 32kB	28 – 32	300 µA/MHz	100 nA

140uA*50MHz=7mA

280uA*200MHz=56mA

300uA*16MHz=4,8mA

*Extraído de : Choosing a MCU for your next design; 8 bit or 32 bit? Ingar Fredriksen, Atmel.

Mercado actual

- KIT DE DESARROLLO: ARDUINO NANO
- MCU: AVR Atmega328p



\$6000 - \$15000

Especificaciones:

- AVR 8 bits CPU
- RISC: 131 instrucciones
- 16MHz clock (0.37 DMIPS/MHz)
- Pipeline 2 etapas (Fetch - Execute) 0.53 coremark/MHz
- ALU 8 bits con Multiplicación 8x8 en 2 ciclos (con y sin signo)
- Hardvard:
 - Program Counter de 14bits (16K words)
 - Instrucción de longitud fija: 16 y 32 bits
 - Ejecución en 1,2 y 3 ciclos de reloj
- Arquitecturas basadas en registros:
 - 32 Registros CPU de 8 bits
 - 3 registros de 16 bits (X,Y,Z)
- 21 fuentes de interrupción vectorizadas
- Modo de bajo consumo SLEEP

Mercado actual

- KIT DE DESARROLLO: BluePill board
- MCU: STM32F103C8T6

Especificaciones:

- 32-bit ARM Cortex-M3 (1.25 DMIPS/MHz)
- Frecuencia de 72 MHz
- 64 KB de Flash
- 20 KB de SRAM 1,50 CoreMark/MHz
- RTC integrado c/ entrada de batería de respaldo
- Modo Sleep, Stop y Standby
- 26 GPIO, la mayoría tolerantes a 5V
- Interrupciones en todas las I/O
- 2 A/D de 12-bit de 1 us, (10 entradas analógicas)
- 7 temporizadores
- 2 interfaces I2C
- 3 interfaces USART
- 2 interfaces SPI
- Interfaz CAN
- Micro USB para alimentación y comunicaciones
- Se puede depurar con SWD (con ST-Link V2)



\$4000 - \$10000

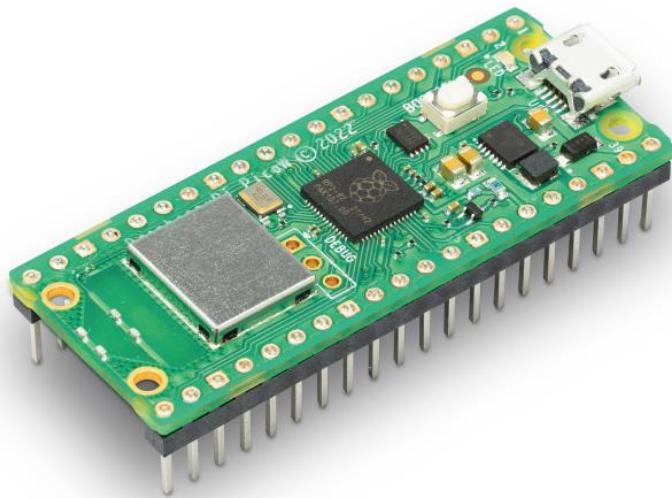
+ Programador \$10000

Mercado actual

- KIT DE DESARROLLO: Raspberry Pi Pico W
- MCU: RP2040

Especificaciones:

- Dual 32 bits ARM Cortex-M0+ (1,7 CoreMark/MHz)
- Frecuencia de operación 133MHz
- 264kB on-chip SRAM
- 2MB of off-chip Flash memory - QSPI bus
- DMA controller
- 30 GPIO pins, 4 of which can be used as analogue inputs
- 2 UARTs
- 2 SPI controllers
- 2 I2C controllers
- 16 PWM channels
- USB 1.1 controller
- 8 PIO state machines



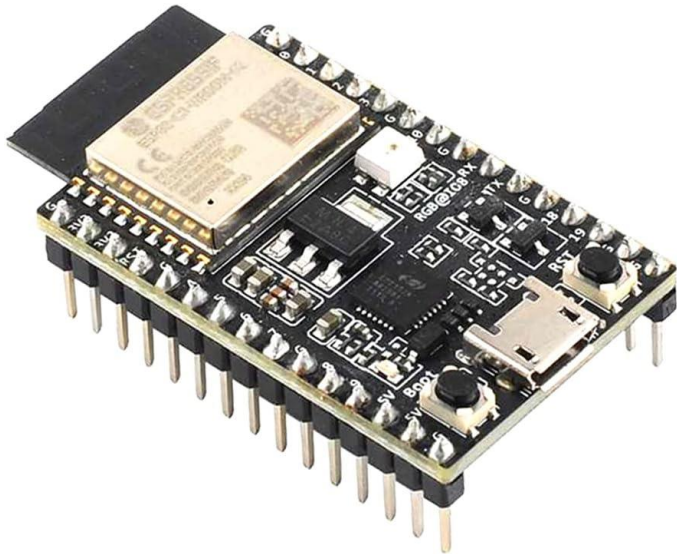
\$11000 - \$25000

Mercado actual

- KIT DE DESARROLLO: ESP32-C3-DevKitC
- SoC: ESP32-c3

Especificaciones:

- 32-bit RISC-V single-core MCU (2.55 CoreMark/MHz)
- Frecuencia máx. 160MHz
- 384 KB of ROM
- 4MB off-chip FLASH
- 400 KB of SRAM
- 8 KB of RTC SRAM
- 22 GPIOs (7 GPIOs need for flash)
- 2 ADC 12-bit (up to 6 channels)
- 1 I2C interface
- 1 I2S interface
- 3 SPI interface
- 2.4 GHz Wi-Fi
- Bluetooth®Low Energy (Bluetooth 5 LE)
- Security: AES, SHA, RSA, RNG, Secure Boot, Flash Encryption



\$25000 - \$125000